

**ARSITEKTUR SISTEM PELATIHAN
LARGE LANGUAGE MODEL TERDISTRIBUSI
BERBASIS *LOG-STRUCTURED STORAGE*
DAN *SPARSE GRADIENT AGGREGATION***

Oleh

**Juan Sohuturon Arauna Siagian
18222086**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Mei 2026**

DAFTAR ISI

DAFTAR ISI	iii
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	2
I.3 Tujuan	3
I.4 Batasan Masalah	3
I.5 Metodologi	4
I.6 Sistematika Penulisan	5

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan *Large Language Model* (LLM) modern didorong oleh peningkatan eksponensial pada jumlah parameter dan volume data pelatihan. Berdasarkan *scaling laws* (Kaplan dkk. 2020), performa model LLM meningkat secara prediktif seiring dengan eskalasi komputasi, ukuran dataset, dan jumlah parameter. Lebih lanjut, analisis empiris mengenai pelatihan *compute-optimal* (Hoffmann dkk. 2022) memperbarui paradigma ini dengan membuktikan bahwa untuk mengoptimalkan komputasi, ukuran model dan jumlah token pelatihan harus diskalakan secara proporsional. Implikasinya, LLM masa kini harus dilatih menggunakan triliunan token, yang secara fundamental menghasilkan beban komputasi masif dan melampaui kapasitas perangkat keras tunggal.

Menghadapi eskalasi kebutuhan komputasi tersebut, adopsi arsitektur pelatihan terdistribusi (*distributed training*) menjadi suatu keharusan untuk memfasilitasi paralelisasi beban kerja. Namun, infrastruktur terdistribusi *State of the Art* (SOTA) saat ini, seperti Megatron-LM (Shoeybi dkk. 2019), PyTorch FSDP (Zhao dkk. 2023), dan arsitektur keluarga ZeRO (Rajbhandari dkk. 2021), dihadapkan pada dua tantangan performa (*bottleneck*) utama yang saling beririsan:

- **Saturasi Komunikasi Jaringan:** Sinkronisasi gradien antar-*node* yang padat (*dense synchronization*) memicu komunikasi data masif yang seringkali melampaui kapasitas *bandwidth* interkoneksi GPU. Meskipun kompresi gradien tradisional telah diusulkan (Lin dkk. 2017), implementasinya pada LLM yang sangat bergantung pada *optimizer* kompleks seperti Adam mengharuskan sinkronisasi *state* yang tetap membebani jaringan (Tang dkk. 2021).
- **Latensi *Input/Output* (I/O):** Untuk mengatasi keterbatasan memori GPU selama pelatihan, arsitektur modern mengandalkan mekanisme *memory offloading* dan penyimpanan *state* model (*checkpointing*) ke media penyimpanan sekunder (Rajbhandari dkk. 2021). Pemindahan tensor model berukuran masif secara asinkron dan terfragmentasi ini menghasilkan beban operasi tulis acak (*random writes*) intensif yang memicu latensi tinggi dan gagal ditangani secara efisien oleh sistem file konvensional.

Akibatnya, optimalisasi parsial yang hanya berfokus pada salah satu lapisan (jaringan atau penyimpanan) tidak akan menghasilkan utilisasi perangkat keras yang maksimal. Siklus komputasi GPU seringkali terhenti (*compute stall*) akibat hambatan operasional (*overhead*) dari lapisan yang tidak teroptimasi.

Merespons kesenjangan tersebut, diperlukan sebuah pendekatan holistik yang mampu mengeliminasi kedua hambatan infrastruktur tersebut secara simultan. Penelitian ini mengusulkan "Arsitektur Sistem Pelatihan LLM Terdistribusi Berbasis *Log-Structured Storage* dan *Sparse Gradient Aggregation*". Secara arsitektural, pendekatan *Sparse Gradient Aggregation* diimplementasikan untuk mereduksi *overhead* jaringan secara drastis dengan membatasi sinkronisasi komunikasi hanya pada elemen gradien yang paling signifikan. Sebagai sistem yang terintegrasi, metode ini dipadukan dengan implementasi *Log-Structured Storage* di tingkat I/O, yang secara intrinsik mengonversi operasi tulis acak menjadi operasi tulis sekuensial (*sequential appends*). Melalui integrasi kedua mekanisme ini, penelitian ini bertujuan untuk merancang sebuah sistem yang dapat memaksimalkan *throughput* I/O sekaligus meminimalkan beban jaringan, sehingga menghasilkan performa pelatihan terdistribusi yang tangguh, efisien, dan memiliki skalabilitas tinggi.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana mereduksi beban komunikasi jaringan (*network overhead*) akibat sinkronisasi gradien yang padat secara efisien pada pelatihan *Large Language Model* (LLM) terdistribusi menggunakan metode *Sparse Gradient Aggregation*?
2. Bagaimana mengatasi tingginya latensi I/O dan fragmentasi operasi tulis acak (*random writes*) pada media penyimpanan sekunder saat proses *memory offloading* dengan mengimplementasikan sistem *Log-Structured Storage*?
3. Bagaimana mengintegrasikan mekanisme *Sparse Gradient Aggregation* dan *Log-Structured Storage* ke dalam sebuah arsitektur pelatihan terdistribusi yang holistik guna mengeliminasi penghentian komputasi GPU (*compute stall*)?
4. Bagaimana tingkat efisiensi, skalabilitas, dan performa arsitektur yang diusulkan jika dibandingkan dengan infrastruktur pelatihan terdistribusi *State of the Art* (SOTA) seperti DeepSpeed atau PyTorch FSDP?

I.3 Tujuan

Tujuan utama dari tugas akhir ini adalah mengimplementasikan dan mengevaluasi arsitektur sistem pelatihan *Large Language Model* terdistribusi yang menggabungkan *Log-Structured Storage* dan *Sparse Gradient Aggregation*. Adapun kriteria keberhasilan dari tugas akhir ini meliputi:

1. **Reduksi *Overhead Jaringan*:** Tercapainya penurunan volume data yang ditransmisikan dan waktu sinkronisasi antar-*node* secara terukur melalui implementasi *Sparse Gradient Aggregation*, tanpa menyebabkan degradasi yang signifikan pada konvergensi model.
2. **Peningkatan *Throughput I/O*:** Berkurangnya latensi pada proses *memory offloading* dan *checkpointing* melalui konversi operasi tulis acak menjadi operasi sekuensial menggunakan *Log-Structured Storage*, yang dibuktikan dengan peningkatan utilisasi *bandwidth* penyimpanan sekunder (seperti NVMe/SSD).
3. **Minimalisasi *Compute Stall*:** Meningkatnya utilisasi komputasi GPU secara keseluruhan selama proses pelatihan, sebagai hasil dari eliminasi hambatan (*bottleneck*) silang antara lapisan jaringan dan sistem I/O.
4. **Keunggulan *Benchmarking*:** Terbuktinya efisiensi arsitektur yang diusulkan melalui peningkatan metrik performa pelatihan (seperti *throughput* token per detik) ketika dibandingkan secara empiris dengan kerangka kerja *State of the Art* (SOTA) yang ada (misalnya DeepSpeed atau PyTorch FSDP).

I.4 Batasan Masalah

Untuk menjaga agar ruang lingkup penelitian tetap terarah dan evaluasi dapat dilakukan secara terukur, batasan masalah yang ditetapkan dalam tugas akhir ini adalah sebagai berikut:

1. **Fokus Evaluasi Kinerja Sistem:** Penelitian ini berfokus pada metrik kinerja infrastruktur sistem (seperti *throughput* token per detik, utilisasi *bandwidth* I/O, dan latensi jaringan). Evaluasi terhadap kualitas bahasa model yang dilatih (metrik akurasi NLP murni) dibatasi hanya pada pembuktian konvergensi awal (*loss degradation*), bukan untuk mencapai akurasi *State of the Art* (SOTA).
2. **Arsitektur dan Skala Model:** Pengujian arsitektur sistem dibatasi pada model bahasa berbasis arsitektur *Transformer decoder-only* dengan rentang jumlah parameter berskala menengah (misalnya, rentang 1 hingga 7 miliar parameter) yang masih merepresentasikan karakteristik LLM modern.

3. **Lingkungan Perangkat Keras:** Evaluasi dilakukan pada kluster GPU dengan interkoneksi jaringan standar (seperti PCIe atau Ethernet/RoCE) dan media penyimpanan sekunder berbasis *Solid State Drive* (SSD) NVMe. Penelitian tidak mencakup optimasi perangkat keras khusus non-GPU (seperti TPU) atau sistem interkoneksi eksotis tingkat superkomputer (seperti topologi *Torus* khusus).
4. **Cakupan Implementasi dan Bahasa Pemrograman:** Implementasi sistem *Log-Structured Storage* dan *Sparse Gradient Aggregation* dibangun menggunakan bahasa pemrograman C dan C++ untuk menjamin kontrol tingkat rendah (*low-level control*) terhadap manajemen memori dan I/O (*system calls*). Pembandingan (*benchmarking*) performa akan diukur terhadap *baseline* kerangka kerja tingkat tinggi (seperti PyTorch FSDP atau DeepSpeed), dan dibatasi hanya pada fase pelatihan (*training phase*).

I.5 Metodologi

Tahapan pelaksanaan penelitian ini dirancang secara sistematis untuk menyelesaikan rumusan masalah, yang meliputi langkah-langkah berikut:

1. **Studi Literatur:** Melakukan penelusuran pustaka mendalam terkait arsitektur *Transformer*, *scaling laws* pada LLM, mekanisme *memory offloading* pada kerangka kerja terdistribusi SOTA (seperti DeepSpeed/ZeRO dan FSDP), teknik kompresi gradien, serta prinsip dasar sistem file berbasis *log-structured*.
2. **Analisis dan Profiling Sistem:** Melakukan *profiling* performa pada *baseline* sistem pelatihan LLM terdistribusi konvensional untuk mengidentifikasi dan mengkuantifikasi metrik *bottleneck* secara empiris, khususnya pada latensi jaringan (*bandwidth* interkoneksi GPU) dan latensi media penyimpanan sekunder (IOPS dan *throughput* tulis acak).
3. **Perancangan Arsitektur:** Merancang integrasi holistik yang terdiri dari dua subsistem utama: subsistem *Sparse Gradient Aggregation* pada lapisan komunikasi jaringan, dan subsistem abstraksi *Log-Structured Storage* di tingkat I/O untuk menangani operasi persistensi dan *checkpointing*.
4. **Implementasi Tingkat Sistem:** Mengembangkan basis kode arsitektur usulan menggunakan bahasa pemrograman C dan C++. Tahap ini mencakup pemrograman tingkat sistem untuk manajemen I/O langsung pada media penyimpanan (*Log-Structured Storage*) dan antarmuka pemrograman paralel/jaringan tingkat rendah untuk menangani *Sparse Gradient Aggregation*.

5. **Evaluasi dan Pengujian:** Melakukan pengujian eksperimental sistem usulan (komparasi performa terhadap *baseline*). Evaluasi difokuskan pada metrik waktu per iterasi (*step time*), *throughput* token per detik, utilisasi *bandwidth* (jaringan dan I/O), serta pengukuran *validation loss* untuk memastikan stabilitas konvergensi model.
6. **Penarikan Kesimpulan:** Menyusun laporan komprehensif berdasarkan analisis data hasil uji eksperimental dan merumuskan saran konstruktif untuk pengembangan penelitian di masa depan.

I.6 Sistematika Penulisan

Laporan penelitian ini disusun dengan sistematika penulisan sebagai berikut:

1. **Bab I Pendahuluan:** Berisi latar belakang, rumusan masalah, tujuan penelitian, batasan masalah, metodologi pelaksanaan, dan sistematika penulisan laporan tugas akhir.
2. **Bab II Studi Literatur:** Membahas kajian teori dan literatur terkait yang menjadi landasan penelitian, mencakup arsitektur LLM, strategi paralelisasi model dan data, optimasi memori terdistribusi, variasi algoritma kompresi gradien, serta paradigma penyimpanan persisten berkinerja tinggi.
3. **Bab III Analisis Sistem:** Menguraikan hasil *profiling* terhadap infrastruktur pelatihan *baseline*, pembuktian empiris atas hambatan kinerja (*compute stall*) akibat latensi I/O dan saturasi jaringan, serta formulasi kebutuhan untuk arsitektur baru.
4. **Bab IV Perancangan Arsitektur:** Menjelaskan desain konseptual dari sistem yang diusulkan, memuat modifikasi alur komunikasi data melalui *Sparse Gradient Aggregation* dan transformasi mekanisme I/O memori menggunakan *Log-Structured Storage*.
5. **Bab V Implementasi:** Menguraikan rincian teknis dari proses perakitan subsistem menggunakan bahasa C/C++, antarmuka pemrograman sistem (*system calls*) yang digunakan, manajemen perangkat keras keras/GPU, serta algoritma inti yang dikembangkan.
6. **Bab VI Evaluasi dan Pengujian:** Menjabarkan metodologi eksperimen, parameter *benchmarking*, penyajian data komparasi antara sistem usulan dan sistem SOTA, serta analisis keberhasilan berdasarkan metrik performa komputasi dan model.
7. **Bab VII Kesimpulan dan Saran:** Menutup laporan dengan kesimpulan akhir yang menjawab rumusan masalah penelitian, beserta rekomendasi praktis untuk optimasi infrastruktur pelatihan LLM selanjutnya.

DAFTAR PUSTAKA

- Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, dkk. 2022. “An empirical analysis of compute-optimal large language model training”. Dalam *Advances in Neural Information Processing Systems*, disunting oleh S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, dan A. Oh, 35:30016–30030. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf.
- Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, dan Dario Amodei. 2020. “Scaling Laws for Neural Language Models”. *CoRR* abs/2001.08361. arXiv: 2001.08361. <https://arxiv.org/abs/2001.08361>.
- Lin, Yujun, Song Han, Huizi Mao, Yu Wang, dan William J. Dally. 2017. “Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training”. *CoRR* abs/1712.01887. arXiv: 1712 . 01887. <http://arxiv.org/abs/1712.01887>.
- Rajbhandari, Samyam, Olatunji Ruwase, Jeff Rasley, Shaden Smith, dan Yuxiong He. 2021. “ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning”. *CoRR* abs/2104.07857. arXiv: 2104.07857. <https://arxiv.org/abs/2104.07857>.
- Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, dan Bryan Catanzaro. 2019. “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism”. *CoRR* abs/1909.08053. arXiv: 1909.08053. <http://arxiv.org/abs/1909.08053>.
- Tang, Hanlin, Shaoduo Gan, Ammar Ahmad Awan, Samyam Rajbhandari, Conglong Li, Xiangru Lian, Ji Liu, Ce Zhang, dan Yuxiong He. 2021. “1-bit Adam: Communication Efficient Large-Scale Training with Adam’s Convergence Speed”. *CoRR* abs/2102.02888. arXiv: 2102 . 02888. <https://arxiv.org/abs/2102.02888>.
- Zhao, Yanli, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, dkk. 2023. *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. arXiv: 2304.11277 [cs.DC]. <https://arxiv.org/abs/2304.11277>.

